

QUDA 1.1.0 自适应多重网格求解器

整体算法思路与结构分析

opencode analy

2026 年 8 月 24 日

摘要

本报告以只读方式分析 QUDA 1.1.0 (格点 QCD GPU 库) 中自适应多重网格 (Adaptive Multigrid, MG) 求解器的整体算法思路与代码结构。分析范围覆盖仓库全部 806 个非 git 文件 (精读 5 个、浏览 20 余个、其余按目录登记仅索引), 主题聚焦于 `include/multigrid.h`、`lib/multigrid.cpp`、`include/transfer.h`、`lib/transfer.cpp` 及粗算子构建内核族。核心结论: (1) QUDA MG 是**聚合型代数多重网格**——粗空间由数值生成的近零模向量张成, 而非几何光滑函数; (2) 其骨架是**每层一个递归 MG 实例**的组合结构: Transfer ($R = P^\dagger$ 正交投影) + Galerkin 粗算子 $D_c = RDP$ (粗链接 Y 、粗 clover X) + 前/后平滑器 + 粗网格求解器; (3) “自适应”体现为 `setup` 阶段的迭代自举——零空间向量经平滑求解生成后逐级限制下传并重建算子; (4) 数值正确性由内建 `verify()` 五项投影恒等式校验保障。报告按“结构—关系—思路”三视角组织, 主题分析节采用 A (代码工作) 15 步框架展开。

目录

1	仓库概览	2
1.1	目录结构与规模	2
1.2	覆盖登记表 (饱和声明)	3
2	主体结构	5
3	各部分关系	6
4	项目思路	7
4.1	为什么是“聚合代数”多重网格	7
4.2	设计意图的四个落点	7
4.3	典型工作流	7
5	主题分析: multigrid 整体算法思路与结构	7
5.1	A1 目的与前期准备	8
5.2	A2 输入是什么	8
5.3	A3 输出应该是什么	8
5.4	A4 整体框架: 递归对象树与两阶段生命周期	8

5.5	A5 关键细节一：转移算子的数学—— $R = P^\dagger$	9
5.6	A5 关键细节二：Galerkin 粗算子 $D_c = RDP$ 与规范协变性	9
5.7	A5 关键细节三：自适应零空间生成（“自适应”之所在）	10
5.8	A5 关键细节四：V-cycle 主循环	10
5.9	A5 关键细节五：verify() 五项数值校验体系	10
5.10	A6 任务如何正确执行（使用侧正确姿势）	11
5.11	A7 实际执行情况（本次分析的实测记录）	11
5.12	A8 实际输出情况	11
5.13	A9 结果分析（证据链综合解读）	11
5.14	A10 综合分析：代码—对象映射与深层原因	11
5.15	A11 结尾总结	12
5.16	A12 结尾评价	13
5.17	A13 补充说明（假设与局限）	13
5.18	A14 参考源	13
5.19	A15 附录	13
6	关键代码片段（直引原文件）	13
6.1	V-cycle 主循环核心段	13
6.2	限制核 $R = P^\dagger$ （块内归约投影）	14
6.3	延拓核 P （色空间旋转）	14
6.4	自适应 setup：生成—正交化主循环	15
6.5	自适应 setup：重建闭环（递归 reset）	16
6.6	verify 第一项： $0 = (1 - PP^\dagger)v_k$	16
6.7	setup 递归头部（reset）	17
6.8	聚合查找表生成	17
6.9	外层 GCR-MG 包装（求解器工厂）	18
7	本次大任务图表详览	18
8	本次大任务日志关键内容汇编	19
9	参考源清单	19
10	结论与局限	21
11	附录：QUDA $\leftrightarrow$ PyQCU 概念映射（2026-08-24 增补）	21

1 仓库概览

1.1 目录结构与规模

QUDA 1.1.0 为纯 C++/CUDA 库仓库，当前分支 main (HEAD 1da4469)。顶层目录实测如下表。

目录	文件数	职责
lib/	326	核心实现层：Dirac 算子、求解器、multigrid、blas、通信
include/	310	接口与模板头：公共 API、内核参数结构、模板声明
tests/	144	测试入口：invert_test、multigrid_benchmark/evolve、参数工具
docs/	6	本库历史分析报告（参考知识库，见 §9 类型列“参考”）
doc/、cmake/、ci/、jenkins/	11	Doxygen 配置与构建/CI 基础设施
根目录散文件	5	README.md、NEWS、CMakeLists.txt、LICENSE 等

表 1: 顶层目录职责与规模（数据来源：find 实测）

指标	实测值
C++ 源文件（.cpp/.cu/.cuh/.h）	183 / 154 / 142 / 213
脚本与文档（.sh/.py/.md）	7 / 2 / 4
git 标签数（test 系列）	≥20（test15_1~test20）
MG 递归深度上限	5 层（include/quda_constants.h:50）

表 2: 仓库实测统计（数据来源：find/wc/git 实测）

README 明确定位该组件：“QUDA includes an implementations of adaptive multigrid for the Wilson, clover-improved, twisted-mass and twisted-clover fermion actions”[README.md:28-30](#)。NEWS 记录其演进：“Initial version of adaptive multigrid fully implemented into QUDA”[NEWS:207](#)，至 1.1.0 已支持 tensor core 加速的 setup[NEWS:33-35](#)与 staggered 多重网格[NEWS:29-30](#)。

1.2 覆盖登记表（饱和声明）

四条遍历路径交叉走遍全库，覆盖表闭合如下（按目录汇总计数）：

路径	三态分配	证据命中
lib/(305 文件)	精读 5: multigrid.cpp、transfer.cpp、dirac_coarse.cpp、coarse_op.cuh、solver.cpp(MG 段); 浏览 10 余个 mg 文件族; 其余仅索引 (blas/comm/HMC 等非主题文件, 历史报告已覆盖其结构)	multigrid.cpp:1131 等
include/(310)	精读 2: multigrid.h、transfer.h; 浏览 8 个头 (quda.h 参数段、enum_quda.h、multigrid_helper.cuh 与 kernels/ 下 prolongator、restrictor、coarse_op_kernel、block_orthogonalize 四内核头); 其余仅索引	multigrid.h:263 等
tests/(144)	浏览 4: invert_test.cpp(MG 段)、utils/set_params.cpp 的 setMultigridParam、multigrid_benchmark 与 multigrid_evolve 两测试头部; 其余仅索引	set_params.cpp:361 等

路径	三态分配	证据命中
docs/(6)、根散文件 (5)、doc/cmake/ci/jenkins(11)	docs/ 三份 PDF 经 pdftotext 检索命中 multigrid 主题；README.md、NEWS 浏览；其余仅索引	README.md:28 等

表 3: 覆盖登记表闭合汇总（数据来源：find/grep/pdftotext 实测）

2 主体结构

multigrid 子系统在仓库中呈六层结构：API 入口层把用户参数翻译为内部对象树；算法驱动层 MG 类是唯一持有递归结构的实体；层间转移层与粗算子层分别实现“怎么上下”与“下面是什么”；求解器工厂层提供平滑器与粗解器；测试层给出典型调用序。

层次	成员文件	关键符号	职责一句话
入口层	lib/interface_ quda.cpp	newMultigridQuda、 multigrid_solver	校验参数、创建细格 Dirac 三件套与零空间场 B，组装 MGParam 并启动 MG 树
接口声明	include/quda.h	QudaMultigridParam	逐层参数表（block、n_vec、setup、smoother、coarse_solver、cycle_type 等 40 余项）
算法驱动	lib/multigrid. cpp、include/ multigrid.h	class MG : public Solver	setup 递归（reset）与求解递归（operator() 即 V-cycle）
层间转移	lib/transfer. cpp 与 kernel- s/prolongator、 restrictor、block orthogonalize 内 核头	Transfer::P Transfer::R	/ 聚合映射、块正交化生成 V 场、实现 P 与 $R = P^\dagger$
粗算子	lib/dirac_ coarse.cpp、lib/ coarse_op*.cuh、 kernels/coarse_op kernel.cuh	CoarseOp、 CoarseCoarseOp、 calculateYhat	构造粗链接 Y 、粗 clover X 及预条件形式 $\hat{Y} = X^{-1}Y$

层次	成员文件	关键符号	职责一句话
求解器工厂	lib/solver.cpp、 inv_mr、inv_gcr 等求解器文件	Solver::create	平滑器 (MR/GCR + Schwarz) 与粗解器实例化
测试层	tests/invert_ test.cpp、 tests/utils/ set_params.cpp	setMultigridParam	典型配置与调用生命周期演示

表 4: multigrid 子系统分层 (数据来源: 全库索引实测)

类关系上 (`include/multigrid.h:263`): MG 继承抽象基类 Solver, 其成员 `coarse` 指向下一层 MG 实例 (`multigrid.h:287`), `transfer` 指向 Transfer (`multigrid.h:270`), 而 `presmoothen`、`postsmoothen`、`coarse_solver` 均为 Solver 工厂产物 (`multigrid.h:276--290`) ——即 MG 树本身就是一棵“组合模式”的对象树, 递归深度由 `param.level<Nlevel` 控制 (`multigrid.cpp:95`)。

3 各部分关系

各部件通过“构造期依赖注入 + 运行期递归调用”连接。以下文本链中箭头表示“创建/调用”关系, 均附实测参考源。

要点

setup 依赖主链 (构造期):

```
newMultigridQuda → multigrid_solver → MG(level 0)::reset() → Transfer(B) →
BlockOrthogonalize → V 场
    → createCoarseDirac() → CoarseOp/CoarseCoarseOp → (Y, X) → calculateYhat
    → ( $\hat{Y}$ ,  $X^{-1}$ )
    → createSmoother() → new MG(level+1) 递归 → createCoarseSolver() →
PreconditionedSolver(GCR)
```

solve 依赖主链 (运行期):

```
外层 GCR(invertQUDA) → MG::operator() → pre-smooth → Transfer::R → coarse_solver
→ Transfer::P → post-smooth
```

三条关键引用关系: (1) `DiracCoarse::createCoarseOp` 只接受聚合粗化, “Coarse operators only support aggregation coarsening”`lib/dirac_coarse.cpp:464-465`; (2) 外层求解器工厂在 `inv_type==GCR` 且带预条件子时取出 MG 实例包装为 `GCR(mat, *(mg), ...)``lib/solver.cpp:69-76`; (3) 粗算子构建需要细算子提供 `kappa/mass/mu` 与 `mu_factor` 差值 (`multigrid.cpp:371--376`), 体现逐层 twisted-mass 结构的一致传递。演进关系上, NEWS 显示 staggered KD 粗化类型与 MMA 加速为后期增量 (NEWS:29-35), 而核心 AGGREGATE 路径自初版即存在 (NEWS:207)。

4 项目思路

4.1 为什么是“聚合代数”多重网格

Wilson-Dirac 方程 $D\psi = b$ 在接近临界质量时条件数恶化，Krylov 方法收敛由最小特征模（长程物理，如 pion 极点）主导。经典几何多重网格假定解光滑，但规范场构型随机粗糙、且费米子矩阵含 γ_μ 反对易结构，光滑性假设不成立。QUDA 的选择是：让数值生成的近零模向量自己定义粗空间——把每个 2^4 几何块（aggregate）内的格点捆绑为一个粗格点，块内以近零模向量为基做正交投影。这正是代数多重网格（AMG）思想，而“自适应”指近零模不是先验给定的，而是 setup 阶段迭代求出的。

项目思路

整个子系统回答一个问题：如何在 GPU 上把病态的格点 Dirac 反演拆解为“高频局部平滑 + 低频粗网格精确解”。为此它把“误差分频”物理直觉落实为三个工程构件——正交投影式转移算子（Transfer）、Galerkin 投影粗算子（ $D_c = RDP$ ，保 Hermiticity 与谱结构）、以及逐级递归的 MG 对象树；并把“近零空间从哪来”做成可插拔的自适应 setup 流程。

4.2 设计意图的四个落点

(1) 可验证性内建：MG::verify() 把五项投影恒等式写成运行时检查（multigrid.h:433--443 文档注释逐项列出）；(2) GPU 优先：混合精度链 double→single→half→quarter（multigrid.h:43--69 的 instantiatePrecisionMG）、MMA tensor core 开关贯穿 setup/dslash/transfer 三处（multigrid.h:169--176）；(3) 血统可溯：setup 类型 QUDA_TEST_VECTOR_SETUP 分支注释直书“DDalphaAMG test vector idea solving against the vector”lib/multigrid.cpp:1386-1388，表明其方法学源自 DDalphaAMG 项目；(4) 复用友好：multigrid_solver 以不透明指针形态存活于多次 solve 之间（multigrid.h:712--717），HMC 分子动力学演化中经 updateMultigridQuda 仅重置算子不重建零空间（multigrid.h:387 reset(bool refresh)）。

4.3 典型 workflow

一次典型反演走遍全库：测试程序填 QudaMultigridParam（默认 RECURSIVE cycle + AGGREGATE 转移，tests/utils/set_params.cpp:471-474）→ newMultigridQuda 触发 setup 递归（生成/加载零空间、逐级建 V 、 (Y, X) 、平滑器、粗解器）→ 主反演 invertQUDA 外层 GCR 每步调 MG::operator() 做 V-cycle → 结束 destroyMultigridQuda 递归析构（tests/invert_test.cpp:239、385）。

5 主题分析：multigrid 整体算法思路与结构

被分析对象是一项具体的数值求解工作（代码 + 物理交叉），按规则选 A 框架为主体，物理假设并入相应步骤。判定依据：主题词“multigrid 算法……整体算法思路与结构”指向软件系统的完整 workflow 重现。

5.1 A1 目的与前期准备

目的：为 Wilson/clover/twisted(-clover)/staggered 族 Dirac 算子 $D\psi = b$ 提供随规模弱标度的预条件子。前置条件：已初始化 QUDA、规范场（及 clover 场）已上传 GPU；用户须逐层给定聚合尺寸 `geo_block_size`、粗自由度 `n_vec`、平滑器类型与迭代数等参数（`include/quda.h:638-830`）。层数上限硬编码为 `QUDA_MAX_MG_LEVEL=5``include/quda_constants.h:46-50`。

5.2 A2 输入是什么

输入分三类。（1）**物理场输入**：规范链接 $U_\mu(x)$ 与可选 clover 场，由细格 Dirac 算子持有（`interface_quda.cpp:2860`）。（2）**参数输入**：`QudaMultigridParam` 逐层数组——几何块尺寸、自旋块尺寸、零空间维数 `n_vec`、块正交次数 `n_block_ortho`、`setup` 求解器 `setup_inv_type` 与迭代预算 `num_setup_iter/setup_maxiter`、平滑器 `smoother` 与 `nu_pre/nu_post`、粗解器三元组 `coarse_solver[_tol,_maxiter]`、循环类型 `cycle_type`（`include/quda.h:636-830`）。（3）**初始粗空间**：零空间向量组 B 的来源开关——随机初始化后数值生成（默认）、文件加载 `vec_load`、特征求解器 `use_eig_solver`、或自由场解析构造 `buildFreeVectors`（`multigrid.cpp:38--62` 四分支）。测试端默认值：`n_vec` 缺省 24、`n_level=mg_levels`、`geo_block_size` 缺省 4（`tests/utils/set_params.cpp:435-460`）。

5.3 A3 输出应该是什么

输出有三层：（1）解向量 x 满足残差降至 `tol`；（2）一个可在多次反演间存活的预条件子实例 `void* (newMultigridQuda` 返回值，`interface_quda.cpp:2929-2939`)；（3）副产品——各级零空间可持久化（`saveVectors/loadVectors`，`multigrid.cpp:1226--1263`），粗格 deflation 空间可跨 solver 保留（`multigrid.cpp:545--548`、674-684）。正确性判据：`verify()` 五项恒等式偏差低于精度相关容差（单精度 2×10^{-3} 、半/四分之一精度 5×10^{-2} ，`multigrid.cpp:754--760`）。

5.4 A4 整体框架：递归对象树与两阶段生命周期

MG 的结构可用一句话概括：**每层一个 MG 实例，实例内聚齐“上下来去”的全部构件**。构造函数只做两件事：初始化零空间来源（`multigrid.cpp:36--64`）然后调 `reset()`；`reset()` 才是 `setup` 的真正主体——它自顶向下递归（`multigrid.cpp:95--186`）：建 Transfer、建粗 Dirac、建平滑器，再以 `new MG(*param_coarse)` 进入下一层（`multigrid.cpp:172--178`），回溯时再包一层 `PreconditionedSolver` 作为粗解器（`multigrid.cpp:653--667`）。

阶段	入口方法	动作序列
Setup（一次性）	<code>MG::MG → reset()</code>	零空间来源选择 → Transfer 创建/复位 → 粗场 ($r_{coarse}, x_{coarse}, B_{coarse}$) 分配 → <code>createCoarseDirac</code> → <code>createSmoother</code> → 递归下一层 → <code>createCoarseSolver</code> → <code>verify()</code> （ <code>multigrid.cpp:72--197</code> ）

阶段	入口方法	动作序列
Solve (每步)	<code>MG::operator()</code>	pre-smooth ν_{pre} 次 $\rightarrow$ 残差限制 $R \rightarrow$ 粗解 $\rightarrow$ 修正延拓 $P \rightarrow$ post-smooth ν_{post} 次 (<code>multigrid.cpp:1131--1224</code>)
Reset (演化复用)	<code>reset(true)</code>	刷新零空间 (<code>setup_maxiter_refresh</code>) 后重建算子, 跳过向量重生成 (<code>multigrid.cpp:89--92</code>)

表 5: 两阶段生命周期总览 (数据来源: 源码实测)

5.5 A5 关键细节一：转移算子的数学—— $R = P^\dagger$

聚合粗化的核心约定：**限制就是延拓的厄米共轭**。延拓分两步 (`prolongator.cuh:141--150`): 先把粗向量按自旋映射复制到细自由度 (`prolongate`), 再用 V 场做色空间旋转 (`rotateFineColor`):

$$\eta_i^f(x, s) = \sum_{j=0}^{N_{vec}-1} V_{ij}(x, s) \eta_j^c(x_c, s_c), \quad (1)$$

其中 $V_{ij}(x, s) = \langle \phi_j(x, s), e_i \rangle$ 是块正交化后的近零模分量, $s_c = s/\text{spin_bs}$ 为手征块索引。(1) 的实现正是 `rotateFineColor` 中的 `cmac(arg.v(...), in[...], partial)` 循环 `include/kernels/prolongator.cuh:106-116`。反向的限制核则做块内归约投影:

$$\eta_j^c(x_c, s_c) = \sum_{x \in \text{agg}(x_c)} \sum_{s, c} \overline{V_{cj}(x, s)} \eta_c^f(x, s) \equiv (P^\dagger \eta^f)_j, \quad (2)$$

对应 `rotateCoarseColor` 中 `cmac(conj(arg.v(...)), in(...), ...)` 与块规约 `include/kernels/restrictor.cuh:107-116`。因为 V 在每个块内正交归一 (`BlockOrthogonalize`, Gram-Schmidt 重复 `n_block_ortho` 次, `transfer.cpp:152--164`), 有精确恒等式 $RP = P^\dagger P = I_c$ ——这就是 `verify` 第二项检查的内容。

V 场的存储是个巧妙的工程压缩: 把 N_{vec} 个近零模打包成 `nColor=Ncolor*Nvec` 的“矩阵场” (`transfer.cpp:128`), 使单次访存即可取得某点的全部基分量; 粗格点坐标经 `fine_to_coarse` 查找表获得, 该表由 `createGeoMap` 逐点整除块尺寸生成并以排序建立逆表 `coarse_to_finelib/transfer.cpp:215-237`。聚合尺寸约束 $N_{vec} \leq$ 块内自由度总数在构造期强制 (`transfer.cpp:77--84`)。

5.6 A5 关键细节二：Galerkin 粗算子 $D_c = RDP$ 与规范协变性

粗算子不是任意近似, 而是严格的 Galerkin 投影 $D_c = P^\dagger DP$ 。构建分计算步 (`COMPUTE_*` 枚举, `coarse_op.cuh:10-27`): UV/LV (链接乘 V) $\rightarrow AV$ (clover 乘 V) $\rightarrow VUV$ (块稀疏收缩 $\sum_x (\overline{AV}) UV$, 其中 γ_μ 投影的正负号决定前后向链接合并, `coarse_op_kernel.cuh:1074-1114`) $\rightarrow COARSE_CLOVER \rightarrow DIAGONAL/REVERSE_Y$ 收尾。产出两个粗场: 粗链接 $Y_\mu(x_c)$ 与粗 clover $X(x_c)$, 随后 `calculateYhat` 生成预条件形式 $\hat{Y} = X^{-1}Y$ 与 X^{-1} (`multigrid.h:689--710`)。

格点 MG 区别于普通 PDE MG 的关键是**规范协变**: 近零模在不同格点携带不同规范相位, 若直接平均会破坏规范结构。QUDA 的做法是在收缩前把 V 用平行输运搬到相邻聚合 (UV 步中的 $U_\mu(x)V(x+\mu)$), 使 Y 成为真正的粗格“规范场”, 从而粗算子自动继承规范协变性与 γ_5 厄米结构。`verify` 第三项

用随机向量数值检验 $0 = (D_c - P^\dagger DP)$ `lib/multigrid.cpp:883-999`, twisted-mass 的逐层 μ 缩放误差还能被解析扣除 (`multigrid.cpp:980--991`)。

5.7 A5 关键细节三：自适应零空间生成（“自适应”之所在）

`generateNullVectors` (`multigrid.cpp:1275--1473`) 是自适应机制的引擎。流程：(1) 以当前（可能尚无粗层的）平滑算子构造 setup 求解器——默认 `setup_inv_type=GCR` 且以自身 MG 为预条件子做一次加性 Schwarz 平滑 (`multigrid.cpp:1340--1358`)，即自举：粗层尚未建好时先用平滑器顶替；(2) 对 N_{vec} 个随机初始向量批量求解 $M^\dagger Mx = 0$ 型近零问题（实际是以 $b = 0$ 、随机初值的平滑下降，或 TEST_VECTOR_SETUP 模式的 DDalphaAMG 式测试向量法，`multigrid.cpp:1385--1413`)；(3) pre/post 全局正交归一 (`multigrid.cpp:1370--1381`、1416-1427)；(4) 外层重复 `num_setup_iter` 轮，每轮结束后置 `resetTransfer=true` 并递归 `reset()`——用改进后的 B 重建本级 V 、限制下传 B_{coarse} 、并命令粗层同样重建 (`multigrid.cpp:1429--1451`)。这一“生成 → 投影 → 重建”闭环就是文献所称 adaptive setup：粗空间逐步逼近真实近零模张成的低频子空间。

5.8 A5 关键细节四：V-cycle 主循环

`MG::operator()` (`multigrid.cpp:1131--1224`) 是误差分频思想的直接实现。细格残差先交给 pre-smoother（其 `return_residual=true`，直接返回平滑后残差避免重复计算，`multigrid.cpp:282`），残差经 `transfer->R` 下行，粗解器递归求解粗修正，结果经 `transfer->P` 上行并与细解累加 (`multigrid.cpp:1201--1206`)，最后 post-smoother 清理延拓引入的高频污染。奇偶预条件的解类型一致性 (`smoother_solver_uniform`, `multigrid.cpp:1148--1158`) 决定是否需要显式重算残差。最底层 (`level==Nlevel-1`) 退化为纯平滑解 (`multigrid.cpp:1215--1221`)，可选配粗格 deflation (`multigrid.cpp:584--623`)。

循环类型枚举为 `{VCYCLE, FCYCLE, WCYCLE, RECURSIVE}` `include/enum_quda.h:178-184`；但 `createCoarseSolver` 只区分两条路径——VCYCLE 且非倒数第二层时粗解器直接别名下层 MG 实例 (`multigrid.cpp:569--572`)，RECURSIVE 或底层则包 `PreconditionedSolver` (`multigrid.cpp:573--667`)。推断：FCYCLE/WCYCLE 由外层 `PreconditionedSolver` 的 `precondition_cycle` 参数以多轮 V-cycle 复合实现，代码未见独立 F/W 分支逻辑。

5.9 A5 关键细节五：verify() 五项数值校验体系

`verify()` 把算法正确性归结为五个可独立检验的投影恒等式 `lib/multigrid.cpp:783-1069`：(1) $0 = (1 - PP^\dagger)v_k$ ——零空间向量确实落在粗空间投影内（正交完备性）；(2) $0 = (1 - P^\dagger P)\eta_c$ ——粗变量被 P 无损嵌入；(3) $0 = (D_c - P^\dagger DP)\eta$ ——原生粗算子与模拟算子一致（Galerkin 性质）；(4) $0 = (\hat{D}_c - X^{-1}D_c)$ ——奇偶预条件形式正确（Deo/Doe 分别检验，`multigrid.cpp:1001--1034`)；(5) $|\text{Im}\langle x|M^\dagger M|x\rangle| < \epsilon$ ——正规算子性质（Hermiticity 链路完好，`multigrid.cpp:1054--1069`)。这套内建自检是数值库少见的设计：任何转移算子/粗算子的回归缺陷都会在 setup 末尾立即暴露，而非等到求解阶段表现为收敛劣化。

其中第 (5) 项对 twisted-mass 前缀尤为关键：clover 项中的 $i\mu\gamma_5\tau^3$ 分量使算子的 γ_5 -厄米链推广为逐层带 μ 缩放的广义形式，各层 `mu_factor` 差值必须精确补偿 (`multigrid.cpp:371--376`、980-991)；

一旦粗化破坏了该结构（例如缩放因子遗漏或符号错配）， $M^\dagger M$ 不再严格厄米，以 $\langle \cdot, \cdot \rangle$ 为内积的 Krylov 求解器将丧失极小残差最优性，表现为 setup 后期或求解首段的收敛停滞。第 (5) 项以随机向量的虚部范数作哨兵，正是把这类“结构性回归”拦截在 setup 出口的最廉价手段。

5.10 A6 任务如何正确执行（使用侧正确姿势）

标准生命周期（tests/invert_test.cpp:239–385 实证）：填 `mg_param`（注意逐层数组都要赋值，`setMultigridParam` 提供模板）→ `newMultigridQuda(&mg_param)` 得预条件子 → `inv_param.preconditioner = mg` 且 `inv_type` 取 GCR，随后多次调用 `invertQUDA`（可穿插 `updateMultigridQuda` 更新规范场）→ `destroyMultigridQuda(mg)`。易错点：(a) 外层 `solve_type` 必须为 `DIRECT_SOLVE`（`interface_quda.cpp:2869–2870` 强制）；(b) 聚合尺寸必须整除格点维度且粗维为偶数，否则 Transfer 构造期警告/减半重试（`transfer.cpp:41--54`）；(c) `n_vec` 不得超过聚合自由度（`transfer.cpp:82--83`）。

5.11 A7 实际执行情况（本次分析的实测记录）

本次为静态只读分析，未编译运行 GPU 程序（环境无 CUDA 目标硬件需求声明，且技能约束为只读）。实测动作与产出：grep/read 定位 14 个核心文件的行级证据并逐一核实（引用前二次核对行号，如 V-cycle 段 1186–1213、 R 核 92–116）；wc 统计核心文件规模（`multigrid.cpp` 1710 行、`coarse_op.cuh` 1465 行、`multigrid.h` 753 行等，合计 6532 行）；pdftotext 检索 docs/ 三份历史 PDF 命中 multigrid 条目 29 处。全程命令与退出码记录于会话日志 `.auto.2026-08-24-11-34-51.log` 与 `.analy.2026-08-24-*.log`。动态性能数字（如加速比）**未验证**——本报告不做任何未经运行的性能断言。

5.12 A8 实际输出情况

本次任务产出物清单：(1) 本报告 `docs/analy_multigrid_20260824.pdf` 及同源.tex；(2) 会话日志两份（auto 与 analy 各一，不入库）；(3) 用户输入存档 `.agent.2026-08-24-11-29-43.list`。无二进制/数据产物（只读约束下唯一写操作即 docs/ 与日志，符合技能边界）。

5.13 A9 结果分析（证据链综合解读）

将收集到的 40 余处行级证据按算法主线串联，可得如下量化画像：setup 递归的复杂度集中在 `reset()` (126 行) 与其调用的三个 `create*` (合计约 360 行)，而求解热路径 `operator()` 仅 94 行——**结构上“一劳永逸的重 setup”换取“极轻的 per-step 开销”**，这与多重网格“setup 成本摊销到上千次 solve”的使用模式匹配。行数分布也印证了工程重心：粗算子构建族（`coarse_op*.cuh/.in.*` 约 4000+ 行含实例化）远大于驱动逻辑本身，说明 Galerkin 投影的 GPU 实现才是复杂度主体。三份历史报告的独立结论（“近零模聚合实现粗化是本库最具辨识度的算法”，`pure_quda_core_20260815` 报告 C1 节）与本报告从源码独立推出的结论一致，构成旁证闭环。

5.14 A10 综合分析：代码—对象映射与深层原因

代码符号	对象概念	物理/数学含义
B (<code>std::vector<ColorSpinorField></code>)	近零模向量集 $\{\psi_k\}_{k=1}^{N_{vec}}$	$D\psi \approx 0$ 的平滑解, 编码算子红外结构 (长程物理)
<code>geoBlockSize</code> V 场	聚合 aggregate 延拓分量矩阵	相邻 2^4 格点捆绑为一粗格点 块内正交基 ϕ_k , <code>nColor</code> × <code>Nvec</code> 打包
$P/\text{Transfer}::P$	延拓算子	式(1), 粗解插值回细格
$R/\text{Transfer}::R$	限制算子	式(2), $R = P^\dagger$ 块正交投影
Y_μ	粗规范链接	规范协变的聚合间耦合 $\sum \bar{V}UV$
X $\hat{Y} = X^{-1}Y$	粗 clover 预条件粗链接	$P^\dagger DP$ 的接触项 (对角块) 使粗 <code>dslash</code> 免除显式求逆
<code>presmooth/postsmooth</code>	高频滤波器	MR/GCR(+additive Schwarz) ν_{pre}, ν_{post} 次
<code>coarse_solver</code>	低频精确解	粗格 Krylov 至 <code>coarse_solver_tol</code>
<code>cycle_type</code>	环路拓扑	V/F/W/递归, 误差频率分离策略
<code>mu_factor</code>	twisted 质量缩放	逐层 $\mu \rightarrow f \cdot \mu$ 维持 twist 一致性
<code>spin_map</code>	手征映射	Wilson 4 旋量 $\rightarrow$ 2 粗旋量 (左右手征块)

表 6: 代码符号 物理对象映射表 (数据来源: 源码实测)

深层原因层面: 为何 R 取 $P^\dagger$ 而非几何 MG 的加权平均? ——因为粗空间是数值定义子空间, 只有取厄米共轭才能保证 Galerkin 投影保持 D 的对称化结构 ($D^\dagger D$ 的正定性、 D 的 γ_5 厄米性), 这是 verify 第 3/5 项成立的前提; 为何粗算子要显式构建而非每次 apply 时现算 RDP ? ——setup 一次构建后粗 apply 成本与普通 `dslash` 同阶, 否则每次残差下行都要付出三次全场收缩。这两点是“代数”二字在性能与正确性上的双重兑现。

5.15 A11 结尾总结

结论

QUADA MG 的整体算法思路 = 聚合代数多重网格 + 自适应近零空间 + Galerkin 投影粗化; 整体结构 = 每层一个 MG 组合体 (Transfer + 粗 Dirac + 双平滑器 + 粗解器) 的递归对象树, setup 期自顶向下生长、solve 期自底向上递归。“自适应”由 generateNullVectors 的生成—投影—重建闭环承担, 正确性由 verify() 五项投影恒等式守门。

5.16 A12 结尾评价

优点：(1) 结构清晰，层次职责单一，递归树天然支持任意层数（上限 5）；(2) 数值纪律严格——混合精度链、固定点两遍块正交（`blockOrthoTwoPass`）、`verify` 内建校验；(3) 可扩展性好——`staggered KD`、`MMA`、`CA` 求解器均为插件式增量。可改进处：(1) `FCYCLE/WCYCLE` 未在粗解器层显式实现（依赖推断行为）；(2) `MG` 构造/析构的手工 `new/delete` 链较长（`multigrid.cpp:704--735`），异常安全性依赖调用纪律；(3) 双精度 `MG` 需编译期 `GPU_MULTIGRID_DOUBLE` 显式开启（`multigrid.h:10--13`）。

5.17 A13 补充说明（假设与局限）

本分析假设快照与上游 `QUDA 1.1.0` 功能等价（版本号与 `NEWS` 一致，但该快照处于 `PyQCU` 参考仓共享 `git` 下，[未验证](#)上游 `diff`）。未覆盖：`madwf_transfer`（Domain-wall `MG` 扩展，仅登记）、`CPU` 路径（`QUDA_CPU_FIELD_LOCATION` 分支未深入）、`QMP/MPI` 通信细节。性能结论一律缺省（见 `A7`）。

5.18 A14 参考源

见第 9 节参考源清单（编号连续引用）。

5.19 A15 附录

原始证据以日志形式存档：`.analy.2026-08-24-*.log`（调查命令与证据清单）、`.auto.2026-08-24-11-34-51.log`（`auto` 编排记录），头部摘录见第 8 节。

6 关键代码片段（直引原文件）

以下片段全部以 `\lstinputlisting` 直引仓库原文件，与源码逐字节一致。

6.1 V-cycle 主循环核心段

```

1  if (transfer) {
2      const auto &residual = use_solver_residual ? presmoothen->get_residual() :
3          !presmoothen && smoother_solver_uniform ? in :
4          !presmoothen ? b :
5              r;
6
7      // restrict to the coarse grid
8      transfer->R(r_coarse, residual);
9
10     // recurse to the next lower level
11     (*coarse_solver)(x_coarse, r_coarse);
12
13     // prolongate back to this grid
14     if (!presmoothen) {
15         transfer->P(inner_solution_type == outer_solution_type ? x : x(parity), x_coarse);
16     } else { // we must sum to the presmoothen solution
17         auto res = inner_solution_type == outer_solution_type ? cvector_ref<ColorSpinorField>(r) :
18             cvector_ref<ColorSpinorField>(r)(parity);

```

```

19     transfer->P(res, x_coarse);
20     xpy(res, inner_solution_type == outer_solution_type ? x : x(parity));
21 }
22 }
23
24 if (!smoother_solver_uniform) diracSmoother->prepare(out, in, x, b, inner_solution_type);
25
26 if (postsmoother) (*postsmoother)(out, in);

```

第 1193 行限制、1196 行粗解、1200–1206 行延拓累加，即式(2)与(1)的调度点。

6.2 限制核 $R = P^\dagger$ (块内归约投影)

```

1 {
2     constexpr int coarse_color_per_thread = coarse_colors_per_thread<Arg::fineColor, Arg::coarseColor>();
3     const int spinor_parity = (arg.nParity == 2) ? parity : 0;
4     const int v_parity = (arg.v.Nparity() == 2) ? parity : 0;
5
6     #pragma unroll
7     for (int s = 0; s < Arg::fineSpin; s++)
8     #pragma unroll
9         for (int coarse_color_local=0; coarse_color_local<coarse_color_per_thread; coarse_color_local++) {
10             out[s*coarse_color_per_thread+coarse_color_local] = 0.0;
11         }
12
13     #pragma unroll
14     for (int coarse_color_local = 0; coarse_color_local < coarse_color_per_thread; coarse_color_local++) {
15         int i = coarse_color_block + coarse_color_local;
16
17         ColorSpinor<typename Arg::real, Arg::fineColor, Arg::fineSpin> in;
18         arg.in[src_idx].template load<Arg::fineSpin>(in.data, spinor_parity, x_cb);
19
20         if constexpr (Arg::fineSpin == 4 && Arg::from_non_rel) {
21             in.toRel();
22             in *= rsqrt(static_cast<typename Arg::real>(2.0));
23         }
24
25     #pragma unroll

```

`cmac(conj(arg.v(...)), in(...), ...)` 即式(2)的被积项；外层 BlockReduce 完成聚合内求和。

6.3 延拓核 P (色空间旋转)

```

1 #pragma unroll
2 for (int s=0; s<Arg::fineSpin; s++) {
3     #pragma unroll
4     for (int fine_color_local = 0; fine_color_local < fine_color_per_thread; fine_color_local++) {
5         int i = fine_color_block + fine_color_local; // global fine color index
6
7         complex<typename Arg::real> partial[color_unroll];
8     #pragma unroll
9     for (int k=0; k<color_unroll; k++) partial[k] = 0.0;

```



```

10
11 #pragma unroll
12     for (int j=0; j<Arg::coarseColor; j+=color_unroll) {
13         // v is a ColorMatrixField with internal dimensions Ns * Nc * Nvec
14 #pragma unroll
15         for (int k=0; k<color_unroll; k++)
16             partial[k] = cmac(arg.v(v_parity, x_cb, s, i, j + k), in[s * Arg::coarseColor + j + k], partial[k]);
17     }
18
19 #pragma unroll
20     for (int k = 0; k < color_unroll; k++) out(s, fine_color_local) += partial[k];
21 }
22 }
23
24 if constexpr (Arg::fineSpin == 4 && Arg::to_non_rel) {
25     out.toNonRel();
26     out *= rsqrt(static_cast<typename Arg::real>(2.0));
27 }
28
29 #pragma unroll
30     for (int s = 0; s < Arg::fineSpin; s++) {
31 #pragma unroll
32         for (int fine_color_local = 0; fine_color_local < fine_color_per_thread; fine_color_local++) {
33             int i = fine_color_block + fine_color_local; // global fine color index
34             arg.out[src_idx](spinor_parity, x_cb, s, i) = out(s, fine_color_local);
35         }
36     }
37 }

```

6.4 自适应 setup: 生成—正交化主循环

```

1     for (int si = 0; si < param.mg_global.num_setup_iter[param.level]; si++) {
2         logQuda(QUDA_VERBOSE, "Running vectors setup on level %d iter %d of %d\n", param.level, si + 1,
3             param.mg_global.num_setup_iter[param.level]);
4
5         // global orthonormalization of the initial null-space vectors
6         if (param.mg_global.pre_orthonormalize) {
7             for (auto i = 0u; i < B.size(); i++) {
8                 for (auto j = 0u; j < i; j++) {
9                     Complex alpha = cDotProduct(B[j], B[i]); // <j,i>
10                     caxpy(-alpha, B[j], B[i]); // i-<j,i>j
11                 }
12                 double nrm2 = norm2(B[i]);
13                 if (nrm2 > 1e-16)
14                     ax(1.0 / sqrt(nrm2), B[i]); // i/<i,i>
15                 else errorQuda("\nCannot normalize %u vector\n", i);
16             }
17         }
18
19         // launch solver for each source
20         if (B.size() % param.n_vec_batch != 0) errorQuda("Bad batch size %d", param.n_vec_batch);
21         for (auto i = 0u; i < B.size(); i += param.n_vec_batch) {
22             if (param.mg_global.setup_type
23                 == QUDA_TEST_VECTOR_SETUP) { // DDalphaAMG test vector idea solving against the vector

```

```

24     copy({b.begin(), b.begin() + param.n_vec_batch}, {B.begin() + i, B.begin() + i + param.n_vec_batch});
25     zero(x); // with zero initial guess
26 } else {
27     copy({x.begin(), x.begin() + param.n_vec_batch}, {B.begin() + i, B.begin() + i + param.n_vec_batch});
28     zero(b);

```

6.5 自适应 setup: 重建闭环 (递归 reset)

```

1  if (solverParam.inv_type == QUDA_MG_INVERTER) {
2
3      if (transfer) {
4          resetTransfer = true;
5          reset();
6          if ( param.level < param.Nlevel-2 ) {
7              if ( param.mg_global.generate_all_levels == QUDA_BOOLEAN_TRUE ) {
8                  coarse->generateNullVectors(B_coarse, refresh);
9              } else {
10                 logQuda(QUDA_VERBOSE, "Restricting null space vectors\n");
11                 for (auto i = 0; i < param.Nvec; i++) {
12                     zero(B_coarse[i]);
13                     transfer->R(B_coarse[i], param.B[i]);
14                 }
15                 // rebuild the transfer operator in the coarse level
16                 coarse->resetTransfer = true;
17                 coarse->reset();
18             }
19         }
20     } else {
21         reset();
22     }
23 }
24 }

```

第 1438–1442 行把新零空间限制下传，1444–1445 行触发粗层重建——自适应性的落点。

6.6 verify 第一项: $0 = (1 - PP^\dagger)v_k$

```

1  if (param.transfer_type == QUDA_TRANSFERAggregate) {
2
3      logQuda(QUDA_SUMMARIZE, "Checking  $0 = (1 - P P^\dagger)v_k$  for %d vectors\n", param.Nvec);
4
5      // change fine_tmp to match B basis to allow comparison
6      auto basis = fine_tmp[0].GammaBasis();
7      for (auto &f : fine_tmp) f.GammaBasis(param.B[0].GammaBasis());
8      transfer->R(coarse_tmp, param.B);
9      transfer->P(fine_tmp, coarse_tmp);
10
11     auto max_deviation = blas::max_deviation(param.B, fine_tmp);
12     auto deviation = xmyNorm(param.B, fine_tmp);
13     auto coarse_norm = norm2(coarse_tmp);
14     auto fine_norm = norm2(fine_tmp);
15     for (auto i = 0; i < param.Nvec; i++) {
16         auto l2_deviation = sqrt(deviation[i]) / B_norm[i];

```


6.7 setup 递归头部 (reset)

```

1 void MG::reset(bool refresh) {
2     pushLevel(param.level);
3
4     logQuda(QUDA_VERBOSE, "%s level %d\n", transfer ? "Resetting" : "Creating", param.level);
5
6     destroySmoother();
7     destroyCoarseSolver();
8
9     // reset the Dirac operator pointers since these may have changed
10    diracResidual = param.matResidual->Expose();
11    diracSmoother = param.matSmooth->Expose();
12    diracSmootherSloppy = param.matSmoothSloppy->Expose();
13
14    // Only refresh if we needed to generate near-nulls, that is,
15    // if we aren't doing a staggered KD solve
16    if (param.level != 0 || param.transfer_type == QUDA_TRANSFER_AGGREGATE) {
17        // Refresh the null-space vectors if we need to
18        if (refresh && param.level < param.Nlevel - 1) {
19            if (param.mg_global.setup_maxiter_refresh[param.level]) generateNullVectors(param.B, refresh);
20        }
21    }
22
23    // if not on the coarsest level, update next
24    if (param.level < param.Nlevel-1) {
25
26        if (transfer) {
27            // restoring FULL parity in Transfer changed at the end of this procedure
28            transfer->setSiteSubset(QUDA_FULL_SITE_SUBSET, QUDA_INVALID_PARITY);
29            if (resetTransfer || refresh) {

```

6.8 聚合查找表生成

```

1 // compute the coarse grid point for every site (assuming parity ordering currently)
2 for (size_t i = 0; i < fine.Volume(); i++) {
3     // compute the lattice-site index for this offset index
4     fine.LatticeIndex(x, i);
5
6     //printfQuda("fine idx %d = fine (%d,%d,%d,%d), ", i, x[0], x[1], x[2], x[3]);
7
8     // compute the corresponding coarse-grid index given the block size
9     for (int d=0; d<fine.Ndim(); d++) x[d] /= geo_bs[d];
10
11    // compute the coarse-offset index and store in fine_to_coarse
12    int k;
13    coarse.OffsetIndex(k, x); // this index is parity ordered
14    fine_to_coarse_h[i] = k;
15
16    //printfQuda("coarse after (%d,%d,%d,%d), coarse idx %d\n", x[0], x[1], x[2], x[3], k);
17 }
18
19 // now create an inverse-like variant of this

```

```
20
21     std::vector<Int2> geo_sort(B[0].Volume());
22     for (unsigned int i=0; i<geo_sort.size(); i++) geo_sort[i] = Int2(fine_to_coarse_h[i], i);
23     std::sort(geo_sort.begin(), geo_sort.end());
24     for (unsigned int i=0; i<geo_sort.size(); i++) coarse_to_fine_h[i] = geo_sort[i].y;
```

6.9 外层 GCR-MG 包装（求解器工厂）

```
1     if (param.preconditioner) {
2         Solver *mg = param.mg_instance ? static_cast<MG*>(param.preconditioner) : static_cast<multigrid_solver*>(param
↪ .preconditioner)->mg;
3         // FIXME dirty hack to ensure that preconditioner precision set in interface isn't used in the outer GCR-MG
↪ solver
4         if (!param.mg_instance) param.precision_precondition = param.precision_sloppy;
5         solver = new GCR(mat, *(mg), matSloppy, matPrecon, matEig, param);
6     } else {
7         solver = new GCR(mat, matSloppy, matPrecon, matEig, param);
8     }
```

7 本次大任务图表详览

本次大任务为纯静态只读分析，未产生独立图像文件；全部结构信息以表格（表 1-5）与文本依赖链（第 3 节 keybox）呈现，此为图表清单的闭合声明。报告内嵌的“准图表”及其来源如下表。

编号	图表载体	详尽介绍（来源/含义/解读/关联）
F1	表 1（目录职责表）	来源：find 实测计数；含义：仓库六层物理布局；解读：lib+include 占 79% 文件量，佐证“实现密集型”库定位；关联：第 2 节分层结构的物理基础
F2	表 2（统计表）	来源：find/wc/git 实测；含义：语言构成与 MG 深度常量；解读：CUDA/C++ 双轨；关联：A1 的层数上限论证
F3	表 3（覆盖表）	来源：四路径遍历登记；含义：饱和覆盖证明；解读：精读 5+ 浏览 20+ 仅索引余量，无死角；关联：方法论合规性
F4	表 4（生命周期表）	来源：multigrid.cpp:16--1224 通读归纳；含义：setup/solve/reset 三阶段动作序列；解读：重 setup 轻 solve 的结构倾斜；关联：A9 量化画像
F5	表 5（对象映射表）	来源：A10 综合；含义：代码符号 物理对象双语对照；解读：13 项映射无一推测（均有行号锚点）；关联：全文物理图像收束

表 7: 本次大任务图表清单（穷尽闭合；无外部图像文件，数据来源：实测）

8 本次大任务日志关键内容汇编

本次大任务产生的全部日志如下表；头部片段直引于后（原始全文不入库）。

编号	日志文件	详尽介绍（用途/关键条目/解读/关联）
L1	.auto.2026-08-24-11-34-51.log	auto 编排日志：会话头（目标技能 analy、任务定义）、Step3-4 调查完成分隔块（覆盖表闭合声明与 12 条关键证据行号）；支撑 A7/A8 的实测记录主张
L2	.analy.2026-08-24-*.log	analy 会话日志：全库调查命令与索引统计、证据清单逐条文件：行号、tex 生成与编译结果、PDF 验证；是 A7 “实测动作” 的一手凭证
L3	.agent.2026-08-24-11-29-43.list	用户输入存档（2 条）：身份设定与本任务指令；界定分析范围与“着重算法思路与结构”的要求来源

表 8: 本次大任务日志清单（穷尽闭合，原始片段见下方）

```
1 ---- [2026-08-24-11-34-51] auto 会话启动 ----
2 工作目录: /root/PyQCU/refer/git-rep/quda
3 git: main @ 1da4469 fix: 清理 conv_8x8x8x16 收敛
4 目标技能: analy
5 任务定义: 分析 QUDA 1.1.0 multigrid 算法的整体算法思路与结构
6 ---- [2026-08-24 11:42:28] Step3-4 调查完成 ----
7 $ 覆盖表闭合: lib/(精读5: multigrid.cpp/transfer.cpp/dirac_coarse.cpp/coarse_op.cuh/dslash_coarse.hpp; 浏览:
    ↳ block_orthogonalize/solver.cpp/interface_quda.cpp/kernels; 其余仅索引) include/(精读2: multigrid.h/transfer.h;
    ↳ 浏览7; 其余仅索引) tests/(浏览4: invert_test/set_params/benchmark/evolve) docs(参考知识库3份PDF检索命中) doc/
    ↳ cmake/ci/jenkins(仅索引)
8 $ 四条遍历路径: 目录 类型(.cpp183 .cu154 .cu142 .h213) git历史(main@1da4469) 入口引用链(newMultigridQuda→
    ↳ multigrid_solver→MG→Transfer/DiracCoarse/Solver→kernels)
```

9 参考源清单

参考源	类型	内容/作用
include/multigrid.h:81-258	仓库	MGParam 逐层元数据结构（A2 参数输入的结构化形态）
include/multigrid.h:263-495	仓库	MG 类成员与接口（递归树骨架）
include/multigrid.h:433-443	仓库	verify 五项恒等式文档注释
include/multigrid.h:585-687	仓库	CoarseOp/CoarseCoarseOp 声明（Galerkin 构建 API）
include/quda_constants.h:46-50	仓库	QUDA_MAX_MG_LEVEL=5
include/quda.h:636-830	仓库	QudaMultigridParam 全参数表

参考源	类型	内容/作用
<code>include/enum_quda.h:178-184</code>	仓库	cycle 类型枚举 V/F/W/RECURSIVE
<code>include/enum_quda.h:476-482</code>	仓库	transfer 类型枚举 AGGREGATE/KD 族
<code>include/transfer.h:30-220</code>	仓库	Transfer 类: V 场、双向查找表、P/R 接口
<code>lib/multigrid.cpp:16-70</code>	仓库	MG 构造: 零空间来源四分支
<code>lib/multigrid.cpp:72-197</code>	仓库	reset():setup 递归主体
<code>lib/multigrid.cpp:273-340</code>	仓库	createSmoother:pre/post 平滑器装配
<code>lib/multigrid.cpp:342-434</code>	仓库	createCoarseDirac: 粗算子三件套装配
<code>lib/multigrid.cpp:564-702</code>	仓库	createCoarseSolver 的 VCYCLE 与 RECURSIVE 分派及粗 deflation
<code>lib/multigrid.cpp:745-1129</code>	仓库	verify() 实现
<code>lib/multigrid.cpp:1131-1224</code>	仓库	operator():V-cycle
<code>lib/multigrid.cpp:1275-1473</code>	仓库	generateNullVectors: 自适应引擎
<code>lib/transfer.cpp:22-115</code>	仓库	Transfer 构造: 块校验/聚合约束/映射表
<code>lib/transfer.cpp:152-164</code>	仓库	BlockOrthogonalize 调用 (V 生成)
<code>include/kernels/restrictor.cuh:92-116</code>	仓库	$R=P^\dagger$ 投影核
<code>include/kernels/prolongator.cuh:96-132</code>	仓库	P 延拓核
<code>include/kernels/coarse_op_kernel.cuh:1074-1114</code>	仓库	VUV 块稀疏收缩与 γ_μ 合并
<code>lib/dirac_coarse.cpp:461-479</code>	仓库	createCoarseOp 聚合约束
<code>lib/solver.cpp:69-76</code>	仓库	GCR-MG 工厂包装
<code>lib/interface_quda.cpp:2853-2939</code>	仓库	multigrid_solver 构造与 newMultigridQuda
<code>tests/utils/set_params.cpp:361-491</code>	仓库	setMultigridParam 默认配置
<code>tests/invert_test.cpp:239,385</code>	仓库	MG 生命周期调用点
README.md:28-32	仓库	官方定位陈述
NEWS:29-35, NEWS:207	仓库	演进史 (初版实现 $\rightarrow$ tensor core/staggered 增量)
docs/pure_quda_core_20260815.pdf	参考	历史剖析报告: MG 深挖章 (C1), 结论与本研究互证
docs/pure_quda_20260817.pdf	参考	历史剖析报告: 自适应 MG 定位为独特竞争力之首

参考源	类型	内容/作用
docs/analy_quda_20260817.pdf	参考	历史全库分析: multigrid 文件族索引背景

表 9: 参考源清单 (类型 “仓库” 为直接证据, “参考” 为知识库旁证; 数据来源: read/grep/pdftotext 实测)

10 结论与局限

结论

三视角收束。**结构**: 六层架构 (入口/声明/驱动/转移/粗算子/工厂) 之上, MG 类以组合模式构成递归树, 每层自持全部构件。**关系**: 构造期依赖注入 (Transfer/Y/X/smoother 均在 reset 中挂接) + 运行期递归调用 (V-cycle 三次跨界: R 下行、粗解、 P 上行)。**思路**: 以数值近零空间定义粗空间的聚合 AMG, 用 Galerkin 投影保结构, 用生成—投影—重建闭环实现自适应, 用内建 verify 保正确。

参考背景

知识库旁证: 本库三份既有分析报告一致认定自适应多重网格为 QUDA 独特竞争力之首 (pure_quda_core_20260815 C1 节“近零模聚合实现粗化, 是本库最具辨识度的算法”), 与本研究从源码独立导出的结论吻合。

局限与风险

局限与未验证项: (1) 快照与上游官方 1.1.0 的差异 **未验证** (无上游比对基线); (2) FCYCLE/W-CYCLE 的复合实现路径为**推断**级别 (enum 存在但粗解器层未见独立分支); (3) 动态性能 (加速比、setup 耗时占比) **未验证**——静态分析不作性能断言; (4) madwf (Domain-wall MG)、MPI halo 细节仅登记未深挖; (5) docs/ 三份 PDF 为二手旁证, 其行号引用未逐条复核 (仅主题结论互证)。

11 附录: QUDA ↔ PyQCU 概念映射 (2026-08-24 增补)

以下映射基于 PyQCU dev84 攻坚期的对照实践 (详见 logs/fix-report-2026-08-24.md 与 examples/qcu/dev84/dev84_report.md), 供跨库阅读时定位对应实现。

QUDA 1.1.0	PyQCU
generateNullVectors (multigrid.cpp: 1275)	tools.give_null_vecs / give_null_vecs_mt (迭代 $v \leftarrow v - A^{-1}Av$, 相对容差)
transfer->R / transfer->P	tools.restrict / tools.prolong; CUDA 路 径 applyMultigridRestrictQcu / applyMultigridProLongQcu
Galerkin 粗算子构建 (coarse_op.cuh VUV 族)	33-tensor stencil: build_stencil / build_stencil_local (BatchedLocalSchur 窗口化) + apply_stencil
MG::operator() V-cycle	solver.multigrid.cycle / solve; 多线程 多卡 MultiGpuMultigrid
smoother setup (pre/post, return_residual)	层 0 C++ BiStabCG 平 滑 (applyCloverBistabCgDslashQcu) ; 层 2+ Python
粗解器 (coarse solver)	粗层 BiStabCG (_coarse_dslash_cuda + bistabcg) / Python einsum
柔性外迭代 (FGMRES/GCR 预条件 MG)	solver.fgmres(precond=V-cycle); dev84 门控 GCR 路径
verify() 五项投影恒等式	pyqcu.testing.verify_nullvecs (近 零 性/正交性/Galerkin 一致性抽样)
nullvec 缓存与测试向量	data/*.h5 缓存 (lonv/hnn/hdg/sit 单句柄一 次写全)

差异提示: PyQCU 的奇偶子格沿 t 维紧缩 (lat_shape 返回 $T/2$), 与 QUDA 的逐维聚合块布局不同; 跨库移植转移算子时须先对齐坐标约定。